MLflow en Databricks

Proyecto 1 — Pronóstico de precios de combustible (Gasolineras)

EQUIPO 3

Nancy Gabriela Mazariegos Molina

José Santiago Pereira Alvarado

Brandon Javier Reyes Morales

Giovanni Alejandro Santos Hernández

Alexis Mesías Flores

Hugo Eduardo Rivas Fajardo

Entregable	Enlace
Jupyter Notebook (workspace de Databricks)	Notebook en Databricks
Repositorio de GitHub (código del pipeline)	https://github.com/Equipo3MLOPS/Proyecto1_Gasolineras-.git
Tablero de Canva: funcionalidades MLflow + Databricks	https://canva.link/mbt4j32qhd3n431

1. ¿Qué usos puede tener MLflow para proyectos de Machine Learning?

MLflow es una plataforma de código abierto para gestionar el ciclo de vida completo de un modelo de Machine Learning. En Databricks viene integrado de forma administrada (sin instalar ni hospedar un servidor), conectado con Unity Catalog para gobierno, permisos y linaje. Sus principales usos son:

- **Seguimiento de experimentos (Tracking / Experiments):** registrar en cada corrida los parámetros, métricas, artefactos (gráficas, modelos) y la versión del código y los datos. Permite comparar corridas lado a lado y reproducir cualquier resultado. Con *autologging* (`mlflow.sklearn.autolog()`) se capturan automáticamente los hiperparámetros y métricas de scikit-learn.
- **Calibración de hiperparámetros:** cada combinación de un `GridSearchCV` / `RandomizedSearchCV` (o de Hyperopt/Optuna) queda como corrida hija, lo que facilita elegir el mejor modelo con evidencia.
- **Empaquetado de modelos (MLflow Models):** un formato estándar (*flavors*: sklearn, pyfunc, xgboost, pytorch…) con firma de entrada/salida, ejemplo de entrada y dependencias, para cargar el modelo en cualquier entorno.
- **Registro y versionado (Model Registry en Unity Catalog):** versiones numeradas, alias (p. ej. *Champion* / *Challenger*), descripciones, permisos y linaje hacia los datos y el notebook que generó cada versión.
- **Despliegue (Model Serving):** publicar un modelo registrado como endpoint REST con escalado automático, o usarlo en inferencia batch (`mlflow.pyfunc.spark_udf`) sobre tablas Delta.

- **Feature Store / Feature Engineering en Unity Catalog:** guardar features reutilizables como tablas, y entrenar con `FeatureLookup` para que el modelo recuerde qué features usa y las busque solo en la inferencia (evita *training–serving skew*).
- **Evaluación y monitoreo:** `mlflow.evaluate()` calcula métricas y gráficas estándar; Lakehouse Monitoring vigila deriva (*drift*) de datos y predicciones en producción. Para GenAI ofrece trazas (*Tracing*) y evaluación con jueces LLM.
- **Trabajo en equipo y MLOps:** un solo lugar donde todo el equipo ve los experimentos, y que se integra con Jobs/Workflows y CI/CD (GitHub Actions) para reentrenar y promover modelos de forma automática.

Aplicado a nuestro proyecto: MLflow nos permite comparar un baseline ingenuo ("el precio del próximo período es igual al actual") contra modelos de regresión, registrar el MAE de cada uno, versionar el Pipeline ganador como `precio_combustible` y, más adelante, servirlo para apoyar la decisión de *cuánto y cuándo comprar* combustible.

2. ¿Qué etapas de CRISP-DM puede cubrir esta herramienta?

MLflow se enfoca en la parte "de modelo" del ciclo, pero dentro de Databricks (junto con Unity Catalog, Feature Store, Model Serving y Monitoring) acompaña prácticamente todas las etapas:

Etapa CRISP-DM	Cobertura	Funcionalidad de MLflow / Databricks
1. Business Understanding	Indirecta	Documentar objetivo y métrica de éxito como descripción y *tags* del experimento; no reemplaza el análisis de negocio.
2. Data Understanding	Parcial	Registrar el EDA como artefactos (gráficas, reportes) y los datasets con `mlflow.log_input` (linaje de datos).
3. Data Preparation	Parcial	Feature Store / Feature Engineering en Unity Catalog; el preprocesamiento queda dentro del Pipeline registrado, así se reproduce igual en inferencia.
4. Modeling	**Total**	Tracking de experimentos, autologging, corridas anidadas para calibración de hiperparámetros, comparación de corridas.
5. Evaluation	**Total**	Métricas por corrida, `mlflow.evaluate()`, comparación contra baseline, alias Champion/Challenger en el Model Registry.
6. Deployment	**Total**	Model Registry (versionado y permisos), Model Serving (endpoint REST), inferencia batch y Lakehouse Monitoring para detectar deriva y reentrenar.

En resumen, MLflow cubre de forma **completa** las etapas de **Modeling, Evaluation y Deployment**, y apoya **parcialmente** Data Understanding y Data Preparation (linaje de datos y Feature Store). Business Understanding sigue siendo trabajo del equipo con el negocio.

3. Adaptación del pipeline de scikit-learn con MLflow

El pipeline ETL del proyecto convierte fotos de un surtidor Shell (Guatemala) en un dataset limpio de precios por galón (capas bronze → silver → gold). Sobre el dataset gold se adaptó el Pipeline de scikit-learn (`ColumnTransformer` con `StandardScaler` + `OneHotEncoder` y un regresor) para usar las funcionalidades de **Experiments** y **Models** de MLflow, en dos versiones:

3.1 Jupyter Notebook (Databricks)

- Notebook `notebooks/calibracion_hiperparametros.ipynb`: calibra Ridge, Lasso, ElasticNet, KNN, RandomForest y GradientBoosting con `GridSearchCV` + `KFold`, y RandomForest también con `RandomizedSearchCV`.
- **Experiments / Tracking:** experimento `calibracion_hiperparametros` con una corrida por modelo (`grid_<modelo>`), más `random_RandomForest` y `resumen_comparacion`. En cada una se registran los mejores hiperparámetros, el número de folds, las features, las métricas (`mae_cv_busqueda`, `mae_promedio`, `mae_std`, `r2_promedio`), el dataset (`mlflow.log_input`) y tags (búsqueda, modelo, etapa CRISP-DM).
- La corrida de resumen guarda la figura de comparación y la tabla de evaluación (`log_artifact` y `log_table`).
- **Models / Model Registry:** cada Pipeline se guarda con `mlflow.sklearn.log_model` (con ejemplo de entrada). El de menor MAE se registra como `precio_combustible_calibrado`, se le asigna el alias `champion` con `MlflowClient` y se vuelve a cargar con `models:/precio_combustible_calibrado@champion` para predecir.
- Las corridas se consultan desde código con `mlflow.search_runs` (filtradas y ordenadas por MAE) y se comparan visualmente en la interfaz de MLflow.
- Enlace: [Notebook en el workspace de Databricks]

Fragmento principal:

```
for nombre, pipe in mejores_pipelines.items():
    with mlflow.start_run(run_name=f"grid_{nombre}"):
        mlflow.log_params(params_por_modelo[nombre]["mejores_params"])
        mlflow.log_metrics({"mae_promedio": ..., "r2_promedio": ...})
        mlflow.sklearn.log_model(pipe, name="modelo", input_example=ejemplo_entrada,
            registered_model_name=MODELO_REGISTRADO if nombre == mejor_modelo else None)

MlflowClient().set_registered_model_alias(MODELO_REGISTRADO, "champion", version_campeon)
```

3.2 Archivo de Python (`src/model.py`)

- Etapa `model` del pipeline (`python -m src.pipeline`), también ejecutada por GitHub Actions y Docker.
- Objetivo: pronosticar el precio del *próximo* período por combustible, con **split temporal** (la última fecha es validación) para no filtrar información del futuro.
- Experimento `precio_gasolineras` con dos corridas: `baseline_carry_forward` y `ridge`; se loguean parámetros, la métrica **MAE** y el dataset de entrada (`mlflow.log_input`).
- El Pipeline se guarda con `mlflow.sklearn.log_model` (con ejemplo de entrada) y se registra como modelo `precio_combustible` en el Model Registry.
- Backend de tracking en SQLite (`mlflow.db`); si MLflow no está instalado, la etapa se omite con un aviso en lugar de romper el pipeline.
- Enlace: https://github.com/Equipo3MLOPS/Proyecto1_Gasolineras-.git

Fragmento principal:

```
mlflow.set_experiment("precio_gasolineras")
dataset = mlflow.data.from_pandas(data, name="dataset_precios", targets=TARGET)

with mlflow.start_run(run_name="ridge"):
    mlflow.log_param("alpha", 1.0)
    mlflow.log_input(dataset, context="training")
    pipe = _build_pipeline(); pipe.fit(Xtr, ytr)
    mlflow.log_metric("mae", mean_absolute_error(yva, pipe.predict(Xva)))
    mlflow.sklearn.log_model(pipe, name="modelo_precio",
        input_example=Xtr.head(2), registered_model_name="precio_combustible")
```

4. Enlaces

- **Jupyter Notebook en Databricks:** https://dbc-a56ba7f1-976e.cloud.databricks.com/editor/notebooks/40376267726831?o=7474656889500873
- **Repositorio de GitHub:** https://github.com/Equipo3MLOPS/Proyecto1_Gasolineras-.git
- **Tablero de Canva (MLflow + Databricks):** https://canva.link/mbt4j32qhd3n431